BACKEND OFICINA · RRMAX

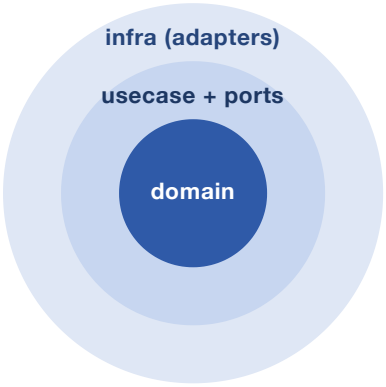
Clean Architecture no módulo de notificações

Como o fluxo de envio e agendamento de notificações foi reorganizado em **domínio → casos de uso → ports → adapters**, explicado com o código real.

Referência: baeldung.com/spring-boot-clean-architecture · Setembro de 2026

1. A ideia em uma página

Clean Architecture organiza o código em camadas concêntricas com uma única regra: **as dependências apontam sempre para dentro**. O centro (domínio e casos de uso) não sabe que existe Spring, JPA, `TaskScheduler` ou HTTP. Tudo que é framework ou infraestrutura fica na borda e se conecta ao centro por meio de **interfaces definidas pelo centro** — os *ports*.



domain — regras puras: normalizar telefone, redigir mensagem, dado a persistir.

usecase — orquestra a regra ("7 dias antes", "sem telefone não envia") usando só interfaces.

port — interfaces que o caso de uso *exige*: enviar, agendar, persistir.

infra — implementações com Spring/JPA/log; e a config que monta tudo.

Estrutura de pacotes criada dentro de `oficina.mecanica.backendOficina`:

árvore de pacotes

```
notificacao/
├── domain/
│   ├── Telefone.java                    regra: destinatário válido
│   ├── MensagemNotificacao.java         regra: textos das mensagens
│   └── NovaNotificacao.java             record: o que vai para o histórico
├── usecase/
│   ├── NotificarFinalizacaoOrdemUseCase.java
│   ├── AgendarLembreteRevisaoUseCase.java
│   ├── OrdemParaNotificar.java          entrada (input)
│   ├── ResultadoNotificacao.java        saída (output)
│   └── port/
│       ├── EnviadorDeMensagem.java      → WhatsApp / SMS / log
│       ├── AgendadorDeLembrete.java     → TaskScheduler
│       └── NotificacaoGateway.java      → banco
└── infra/
    ├── LogEnviadorDeMensagem.java       implementa EnviadorDeMensagem
    ├── TaskSchedulerAgendadorDeLembrete.java  implementa AgendadorDeLembrete
    ├── NotificacaoJpaGateway.java       implementa NotificacaoGateway
    ├── OrdemParaNotificarMapper.java    entidade JPA → input
    └── NotificacaoUseCaseConfig.java    @Configuration: monta os beans
```

2. Como era antes

O `NotificacaoService` original tinha 436 linhas e misturava, na mesma classe, regra de negócio, agendamento, persistência, texto de mensagem, conversão para DTO e exceção HTTP. Basta olhar os imports e os campos para ver o acoplamento:

Service/NotificacaoService.java — linhas 1–56

```java
package oficina.mecanica.backendOficina.Service;

import oficina.mecanica.backendOficina.DTO.NotificacaoDTOResponse;
import oficina.mecanica.backendOficina.DTO.NotificacaoResultadoDTO;
import oficina.mecanica.backendOficina.Model.CanalNotificacao;
import oficina.mecanica.backendOficina.Model.ClienteModel;
import oficina.mecanica.backendOficina.Model.NotificacaoModel;
import oficina.mecanica.backendOficina.Model.OrdemServicoModel;
import oficina.mecanica.backendOficina.Model.StatusNotificacao;
import oficina.mecanica.backendOficina.Model.TipoNotificacao;
import oficina.mecanica.backendOficina.Model.VeiculoModel;
import oficina.mecanica.backendOficina.Repository.NotificacaoRepository;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.scheduling.TaskScheduler;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Service;
import org.springframework.web.server.ResponseStatusException;
import org.springframework.transaction.annotation.Transactional;

import java.time.LocalDateTime;
import java.time.ZoneId;
import java.time.format.DateTimeFormatter;
import java.util.Date;
import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ScheduledFuture;

/**
 * Registra o histórico de notificações da oficina.
 *
 * O envio externo (WhatsApp/SMS) está desativado: as mensagens são montadas,
 * gravadas na tabela `notificacoes` e registradas no log, o que mantém a tela de
 * notificações funcional sem depender de um gateway configurado.
 */
@Service
public class NotificacaoService {

    private static final Logger log = LoggerFactory.getLogger(NotificacaoService.class);
    private static final DateTimeFormatter DATA_PT_BR =
DateTimeFormatter.ofPattern("dd/MM/yyyy");
    private static final String ASSUNTO_FINALIZACAO = "Serviço finalizado";
    private static final String ASSUNTO_REVISAO = "Lembrete de revisão preventiva";

    private final TaskScheduler taskScheduler;
    private final NotificacaoRepository notificacaoRepository;
    private final Map<Long, ScheduledFuture<?>> lembretesAgendados = new ConcurrentHashMap<>
();
    private final Set<Long> lembretesProcessados = ConcurrentHashMap.newKeySet();
```

```
50
51      public NotificacaoService(TaskScheduler taskScheduler, NotificacaoRepository
    notificacaoRepository) {
52          this.taskScheduler = taskScheduler;
53          this.notificacaoRepository = notificacaoRepository;
54      }
55
56      public List<NotificacaoDTOResponse> listar() {
```

Versão anterior (commit bce4af3). Repare: `TaskScheduler`, `NotificacaoRepository`, `ResponseStatusException` e dois mapas de estado em memória convivem com a regra de negócio.

O método que "envia" era um `private` que só escrevia no log, com um comentário avisando que ali seria o ponto de troca quando houvesse um gateway. Em Clean Architecture esse "ponto de troca" vira uma interface — o port `EnviadorDeMensagem` que veremos adiante.

Service/NotificacaoService.java — linhas 233–255

```
233      * Substitui o envio externo: apenas valida o destinatário e registra a mensagem no log.
234      * Quando um gateway de mensagens for contratado, é este o único ponto a ser trocado.
235      */
236     private boolean registrarEnvio(String telefoneCliente, String mensagem, String contexto) {
237         String destinatario = normalizarTelefone(telefoneCliente);
238         if (destinatario == null) {
239             log.warn("Notificação de {} não registrada como enviada: telefone do cliente
    ausente.", contexto);
240             return false;
241         }
242
243         log.info("Notificação de {} registrada (envio externo desativado). Para: {} | Texto:
    {}",
244                 contexto,
245                 destinatario,
246                 mensagem
247         );
248
249         return true;
250     }
251
252     private String normalizarTelefone(String telefone) {
253         String digitos = valorOuPadrao(telefone, "").replaceAll("[^0-9]", "");
254         return digitos.isBlank() ? null : digitos;
255     }
```

O envio era um método privado; trocar de provedor exigia editar a classe de regra.

3. Camada de domínio

Aqui só entra Java puro. Nenhum import de Spring, JPA ou log. São regras que valem independentemente de como a aplicação é executada.

3.1 Telefone — quando existe um destinatário?

notificacao/domain/Telefone.java

```java
package oficina.mecanica.backendOficina.notificacao.domain;

/**
 * Regra de domínio para destinatários: um telefone só é válido para envio
 * quando, após remover a formatação, restam dígitos.
 */
public final class Telefone {

    private Telefone() {
    }

    /** Retorna apenas os dígitos do telefone, ou {@code null} quando não há destinatário válido. */
    public static String normalizar(String telefone) {
        if (telefone == null) {
            return null;
        }

        String digitos = telefone.replaceAll("[^0-9]", "");
        return digitos.isBlank() ? null : digitos;
    }
}
```

Regra: removida a formatação, precisa sobrar pelo menos um dígito. Antes isso estava enterrado em `registrarEnvio`.

3.2 MensagemNotificacao — o que a oficina diz ao cliente

Os textos são regra de negócio (a redação, a saudação com o nome da oficina, como descrever o veículo). Concentrá-los aqui permite que nem o use case nem a infra conheçam o formato.

notificacao/domain/MensagemNotificacao.java — linhas 1–41

```java
package oficina.mecanica.backendOficina.notificacao.domain;

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

/**
 * Textos enviados ao cliente. Concentra a redação das mensagens da oficina
 * para que nenhuma camada externa precise conhecer o formato.
 */
public final class MensagemNotificacao {

    public static final String ASSUNTO_FINALIZACAO = "Serviço finalizado";
    public static final String ASSUNTO_REVISAO = "Lembrete de revisão preventiva";

    private static final DateTimeFormatter DATA_PT_BR =
DateTimeFormatter.ofPattern("dd/MM/yyyy");

    private MensagemNotificacao() {
    }

    public static String finalizacao(String nomeCliente, String modeloVeiculo, String
placaVeiculo) {
        return String.format(
                "%s O serviço do veículo %s foi finalizado pela oficina. " +
                        "A garantia do serviço já está ativa conforme as condições informadas.
Obrigado pela preferência!",
                saudacao(nomeCliente),
                descricaoVeiculo(modeloVeiculo, placaVeiculo)
        );
    }

    public static String lembreteRevisao(String nomeCliente,
                                          String modeloVeiculo,
                                          String placaVeiculo,
                                          LocalDateTime dataRevisao) {
        return String.format(
                "%s Passando para lembrar que a revisão preventiva do veículo %s está prevista
para %s. " +
                        "Recomendamos entrar em contato com a oficina para confirmar o
agendamento.",
                saudacao(nomeCliente),
                descricaoVeiculo(modeloVeiculo, placaVeiculo),
                dataRevisao.format(DATA_PT_BR)
        );
    }
```

Dois textos públicos: finalização de serviço e lembrete de revisão. Recebem só strings e uma data — nada de entidade.

notificacao/domain/MensagemNotificacao.java — linhas 42–71

```
42      private static String saudacao(String nomeCliente) {
43          String nome = valorOuPadrao(nomeCliente, "");
44          return nome.isBlank()
45                  ? "Olá! Aqui é da RRmax oficina mecânica."
46                  : "Olá, " + nome + "! Aqui é da RRmax oficina mecânica.";
47      }
48
49      private static String descricaoVeiculo(String modeloVeiculo, String placaVeiculo) {
50          String modelo = valorOuPadrao(modeloVeiculo, "");
51          String placa = valorOuPadrao(placaVeiculo, "");
52
53          if (!modelo.isBlank() && !placa.isBlank()) {
54              return modelo + " / placa " + placa;
55          }
56
57          if (!placa.isBlank()) {
58              return "placa " + placa;
59          }
60
61          if (!modelo.isBlank()) {
62              return modelo;
63          }
64
65          return "não informado";
66      }
67
68      private static String valorOuPadrao(String valor, String padrao) {
69          return valor != null && !valor.isBlank() ? valor.trim() : padrao;
70      }
71  }
```

Helpers privados: saudação e descrição do veículo com todos os fallbacks (sem modelo, sem placa, sem nada).

3.3 NovaNotificacao — o que vai para o histórico

Um `record` com identificadores e valores. O caso de uso monta este objeto e entrega ao gateway; é o gateway (infra) que sabe transformá-lo em `NotificacaoModel` com as relações JPA.

notificacao/domain/NovaNotificacao.java

```java
package oficina.mecanica.backendOficina.notificacao.domain;

import oficina.mecanica.backendOficina.Model.StatusNotificacao;
import oficina.mecanica.backendOficina.Model.TipoNotificacao;

import java.time.LocalDateTime;

/**
 * Notificação a ser registrada no histórico. É o que o caso de uso entrega
 * ao gateway de persistência — sem entidade JPA, apenas identificadores.
 */
public record NovaNotificacao(
        Long clienteId,
        Long veiculoId,
        Long ordemServicoId,
        TipoNotificacao tipo,
        String assunto,
        String mensagem,
        StatusNotificacao status,
        String telefoneDestino,
        LocalDateTime dataAgendamento,
        LocalDateTime dataEnvio
) {
}
```

Os enums `TipoNotificacao` / `StatusNotificacao` são reutilizados do pacote `Model`: são enums Java puros, sem anotação JPA.

4. Ports — o que o caso de uso exige do mundo externo

Os ports são interfaces **declaradas dentro do pacote do caso de uso**. Isso inverte a dependência: a infra é quem implementa o contrato que o centro definiu, e não o contrário. O caso de uso nunca importa nada de `infra`.

4.1 EnviadorDeMensagem

notificacao/usecase/port/EnviadorDeMensagem.java

```java
package oficina.mecanica.backendOficina.notificacao.usecase.port;

/**
 * Port de saída para o gateway de mensagens (WhatsApp/SMS/log).
 * Trocar o provedor significa apenas trocar a implementação deste contrato.
 */
public interface EnviadorDeMensagem {

    /**
     * @param telefone destinatário já normalizado (somente dígitos)
     * @param mensagem texto a enviar
     * @param contexto descrição curta para rastreabilidade (ex.: "finalização da OS 12")
     * @return {@code true} se o envio foi aceito pelo gateway
     */
    boolean enviar(String telefone, String mensagem, String contexto);
}
```

Um único método. Hoje a implementação escreve no log; amanhã pode ser Twilio — o caso de uso não muda.

4.2 AgendadorDeLembrete

notificacao/usecase/port/AgendadorDeLembrete.java

```java
package oficina.mecanica.backendOficina.notificacao.usecase.port;

import java.time.LocalDateTime;

/**
 * Port de saída para agendamento de tarefas futuras e controle de
 * duplicidade dos lembretes dentro da execução atual da aplicação.
 */
public interface AgendadorDeLembrete {

    boolean estaAgendado(Long ordemServicoId);

    boolean foiProcessado(Long ordemServicoId);

    void marcarProcessado(Long ordemServicoId);

    void agendar(Long ordemServicoId, LocalDateTime quando, Runnable tarefa);
}
```

Abstrai o `TaskScheduler` e o controle de duplicidade (agendado / processado) da execução atual.

4.3 NotificacaoGateway

notificacao/usecase/port/NotificacaoGateway.java

```java
package oficina.mecanica.backendOficina.notificacao.usecase.port;

import oficina.mecanica.backendOficina.notificacao.domain.NovaNotificacao;

import java.time.LocalDateTime;
import java.util.Optional;

/**
 * Port de saída para o histórico de notificações.
 */
public interface NotificacaoGateway {

    /** Registra a notificação e devolve o id gerado, ou vazio se não foi possível persistir. */
    Optional<Long> registrar(NovaNotificacao notificacao);

    /**
     * Atualiza mensagem e data de um lembrete de revisão ainda pendente para a OS.
     * Devolve o id do lembrete atualizado, ou vazio quando não existe pendente.
     */
    Optional<Long> atualizarLembreteRevisaoPendente(Long ordemServicoId, String mensagem, LocalDateTime dataAgendamento);

    /** Marca uma notificação agendada como enviada (com data) ou com erro. */
    void registrarResultadoEnvio(Long notificacaoId, boolean enviada, LocalDateTime dataEnvio);
}
```

Três operações de persistência, descritas em termos do negócio (“registrar”, “atualizar pendente”, “resultado do envio”), não em termos de JPA.

5. Casos de uso

Classes Java comuns — sem `@Service`, sem `@Autowired`. Recebem os ports e um `java.time.Clock` pelo construtor. Entrada e saída são records próprios do caso de uso.

5.1 Entrada e saída

notificacao/usecase/OrdemParaNotificar.java

```java
package oficina.mecanica.backendOficina.notificacao.usecase;

import java.time.LocalDateTime;

/**
 * Dados de entrada dos casos de uso de notificação. Contém apenas o que o
 * fluxo precisa da ordem de serviço, desacoplado da entidade JPA.
 */
public record OrdemParaNotificar(
        Long id,
        Long clienteId,
        String nomeCliente,
        String telefoneCliente,
        Long veiculoId,
        String modeloVeiculo,
        String placaVeiculo,
        LocalDateTime dataProximaRevisao
) {
}
```

A entrada carrega só o que o fluxo precisa da OS. Nada de `OrdemServicoModel`: o caso de uso não conhece JPA.

notificacao/usecase/ResultadoNotificacao.java

```java
package oficina.mecanica.backendOficina.notificacao.usecase;

import java.time.LocalDateTime;

/**
 * Saída dos casos de uso de notificação.
 */
public record ResultadoNotificacao(
        boolean enviada,
        boolean agendada,
        boolean enviadaImediatamente,
        LocalDateTime dataAgendamento,
        String mensagem,
        String aviso
) {

    public static ResultadoNotificacao enviada(String mensagem) {
        return new ResultadoNotificacao(true, false, true, null, mensagem, null);
    }

    public static ResultadoNotificacao enviadaImediatamente(LocalDateTime quando, String mensagem) {
        return new ResultadoNotificacao(true, false, true, quando, mensagem, null);
    }

    public static ResultadoNotificacao agendada(LocalDateTime dataAgendamento, String mensagem) {
        return new ResultadoNotificacao(false, true, false, dataAgendamento, mensagem, null);
    }

    public static ResultadoNotificacao naoEnviada(String aviso) {
        return new ResultadoNotificacao(false, false, false, null, null, aviso);
    }
}
```

Substitui o antigo `NotificacaoResultadoDTO`. As fábricas estáticas nomeiam os resultados possíveis.

5.2 NotificarFinalizacaoOrdemUseCase

Fluxo linear: monta o texto (domínio) → normaliza o telefone (domínio) → envia pelo port → registra pelo port → devolve o resultado. Observe que a decisão “sem telefone não envia” está aqui, não no enviador.

notificacao/usecase/NotificarFinalizacaoOrdemUseCase.java

```java
package oficina.mecanica.backendOficina.notificacao.usecase;

import oficina.mecanica.backendOficina.Model.StatusNotificacao;
import oficina.mecanica.backendOficina.Model.TipoNotificacao;
import oficina.mecanica.backendOficina.notificacao.domain.MensagemNotificacao;
import oficina.mecanica.backendOficina.notificacao.domain.NovaNotificacao;
import oficina.mecanica.backendOficina.notificacao.domain.Telefone;
import oficina.mecanica.backendOficina.notificacao.usecase.port.EnviadorDeMensagem;
import oficina.mecanica.backendOficina.notificacao.usecase.port.NotificacaoGateway;

import java.time.Clock;
import java.time.LocalDateTime;

/**
```

```java
 * Avisa o cliente que o serviço da ordem foi finalizado e registra o histórico.
 */
public class NotificarFinalizacaoOrdemUseCase {

    private final EnviadorDeMensagem enviador;
    private final NotificacaoGateway gateway;
    private final Clock clock;

    public NotificarFinalizacaoOrdemUseCase(EnviadorDeMensagem enviador, NotificacaoGateway gateway, Clock clock) {
        this.enviador = enviador;
        this.gateway = gateway;
        this.clock = clock;
    }

    public ResultadoNotificacao executar(OrdemParaNotificar ordem) {
        String mensagem = MensagemNotificacao.finalizacao(
                ordem.nomeCliente(), ordem.modeloVeiculo(), ordem.placaVeiculo());

        String telefone = Telefone.normalizar(ordem.telefoneCliente());
        boolean enviada = telefone != null
                && enviador.enviar(telefone, mensagem, "finalização da OS " + ordem.id());

        if (ordem.clienteId() != null) {
            gateway.registrar(new NovaNotificacao(
                    ordem.clienteId(),
                    ordem.veiculoId(),
                    ordem.id(),
                    TipoNotificacao.pos_servico,
                    MensagemNotificacao.ASSUNTO_FINALIZACAO,
                    mensagem,
                    enviada ? StatusNotificacao.enviada : StatusNotificacao.erro,
                    ordem.telefoneCliente(),
                    null,
                    enviada ? LocalDateTime.now(clock) : null
            ));
        }

        if (enviada) {
            return ResultadoNotificacao.enviada("Notificação de finalização registrada para o cliente.");
        }

        return ResultadoNotificacao.naoEnviada("Notificação de finalização não registrada: cliente sem telefone cadastrado.");
    }
}
```

`LocalDateTime.now(clock)` em vez de `now()`: o relógio é injetado, então o teste controla o tempo.

5.3 AgendarLembreteRevisaoUseCase — a regra dos 7 dias

É o caso de uso mais rico. A constante `DIAS_ANTECEDENCIA` torna a regra explícita. O método `executar` só decide o caminho; cada caminho fica em um método privado.

notificacao/usecase/AgendarLembreteRevisaoUseCase.java — linhas 1–40

```java
package oficina.mecanica.backendOficina.notificacao.usecase;

import oficina.mecanica.backendOficina.Model.StatusNotificacao;
import oficina.mecanica.backendOficina.Model.TipoNotificacao;
import oficina.mecanica.backendOficina.notificacao.domain.MensagemNotificacao;
import oficina.mecanica.backendOficina.notificacao.domain.NovaNotificacao;
import oficina.mecanica.backendOficina.notificacao.domain.Telefone;
import oficina.mecanica.backendOficina.notificacao.usecase.port.AgendadorDeLembrete;
import oficina.mecanica.backendOficina.notificacao.usecase.port.EnviadorDeMensagem;
import oficina.mecanica.backendOficina.notificacao.usecase.port.NotificacaoGateway;

import java.time.Clock;
import java.time.LocalDateTime;

/**
 * Regra do lembrete de revisão preventiva: o cliente é avisado
 * {@value #DIAS_ANTECEDENCIA} dias antes da data prevista. Se a data já está
 * dentro desse prazo, o lembrete é enviado imediatamente; caso contrário, é
 * registrado como pendente e agendado.
 */
public class AgendarLembreteRevisaoUseCase {

    public static final int DIAS_ANTECEDENCIA = 7;

    private final EnviadorDeMensagem enviador;
    private final NotificacaoGateway gateway;
    private final AgendadorDeLembrete agendador;
    private final Clock clock;

    public AgendarLembreteRevisaoUseCase(EnviadorDeMensagem enviador,
                                         NotificacaoGateway gateway,
                                         AgendadorDeLembrete agendador,
                                         Clock clock) {
        this.enviador = enviador;
        this.gateway = gateway;
        this.agendador = agendador;
        this.clock = clock;
    }

    public ResultadoNotificacao executar(OrdemParaNotificar ordem) {
```

Cabeçalho: a constante `DIAS_ANTECEDENCIA` nomeia a regra; o construtor recebe três ports e o `Clock`.

notificacao/usecase/AgendarLembreteRevisaoUseCase.java — linhas 41–65

```
41          if (ordem.dataProximaRevisao() == null) {
42              return ResultadoNotificacao.naoEnviada("Data de revisão preventiva não
    informada.");
43          }
44
45          Long ordemId = ordem.id();
46          if (ordemId != null && agendador.foiProcessado(ordemId)) {
47              return ResultadoNotificacao.naoEnviada("Lembrete de revisão já processado nesta
    execução.");
48          }
49
50          LocalDateTime dataEnvio = ordem.dataProximaRevisao().minusDays(DIAS_ANTECEDENCIA);
51          LocalDateTime agora = LocalDateTime.now(clock);
52
53          if (!dataEnvio.isAfter(agora)) {
54              return enviarAgora(ordem, agora);
55          }
56
57          if (ordemId != null && agendador.estaAgendado(ordemId)) {
58              return ResultadoNotificacao.agendada(dataEnvio, "Lembrete de revisão já estava
    agendado.");
59          }
60
61          Long notificacaoId = registrarOuAtualizarPendente(ordem, dataEnvio);
62          agendador.agendar(ordemId, dataEnvio, () -> enviarAgendado(ordem, notificacaoId));
63
64          return ResultadoNotificacao.agendada(dataEnvio, "Lembrete de revisão preventiva
    agendado.");
65      }
```

Sequência de guardas: sem data → sai; já processado → sai; dentro do prazo → envia agora; já agendado → sai; senão registra pendente e agenda.

notificacao/usecase/AgendarLembreteRevisaoUseCase.java — linhas 66–109

```java
    private ResultadoNotificacao enviarAgora(OrdemParaNotificar ordem, LocalDateTime agora) {
        String mensagem = mensagem(ordem);
        boolean enviada = enviar(ordem, mensagem);

        if (ordem.clienteId() != null) {
            gateway.registrar(new NovaNotificacao(
                    ordem.clienteId(),
                    ordem.veiculoId(),
                    ordem.id(),
                    TipoNotificacao.revisao_preventiva,
                    MensagemNotificacao.ASSUNTO_REVISAO,
                    mensagem,
                    enviada ? StatusNotificacao.enviada : StatusNotificacao.erro,
                    ordem.telefoneCliente(),
                    null,
                    enviada ? agora : null
            ));
        }

        if (!enviada) {
            return ResultadoNotificacao.naoEnviada("Lembrete de revisão não registrado: cliente sem telefone cadastrado.");
        }

        if (ordem.id() != null) {
            agendador.marcarProcessado(ordem.id());
        }

        return ResultadoNotificacao.enviadaImediatamente(
                agora, "Data da revisão com menos de " + DIAS_ANTECEDENCIA + " dias; lembrete registrado imediatamente.");
    }

    /** Executado pelo agendador na data prevista. */
    private void enviarAgendado(OrdemParaNotificar ordem, Long notificacaoId) {
        boolean enviada = enviar(ordem, mensagem(ordem));

        if (notificacaoId != null) {
            gateway.registrarResultadoEnvio(notificacaoId, enviada, enviada ? LocalDateTime.now(clock) : null);
        }

        if (enviada && ordem.id() != null) {
            agendador.marcarProcessado(ordem.id());
        }
    }
```

`enviarAgora` registra a notificação como *enviada* ou *erro*. `enviarAgendado` é o `Runnable` que o agendador executará na data — ele só conhece o port, não o `TaskScheduler`.

notificacao/usecase/AgendarLembreteRevisaoUseCase.java — linhas 110–151

```java
    private Long registrarOuAtualizarPendente(OrdemParaNotificar ordem, LocalDateTime dataAgendamento) {
        String mensagem = mensagem(ordem);

        if (ordem.id() != null) {
            Long atualizado = gateway
                    .atualizarLembreteRevisaoPendente(ordem.id(), mensagem, dataAgendamento)
                    .orElse(null);
            if (atualizado != null) {
                return atualizado;
            }
        }

        if (ordem.clienteId() == null) {
            return null;
        }

        return gateway.registrar(new NovaNotificacao(
                ordem.clienteId(),
                ordem.veiculoId(),
                ordem.id(),
                TipoNotificacao.revisao_preventiva,
                MensagemNotificacao.ASSUNTO_REVISAO,
                mensagem,
                StatusNotificacao.pendente,
                ordem.telefoneCliente(),
                dataAgendamento,
                null
        )).orElse(null);
    }

    private boolean enviar(OrdemParaNotificar ordem, String mensagem) {
        String telefone = Telefone.normalizar(ordem.telefoneCliente());
        return telefone != null
                && enviador.enviar(telefone, mensagem, "lembrete de revisão da OS " + ordem.id());
    }

    private String mensagem(OrdemParaNotificar ordem) {
        return MensagemNotificacao.lembreteRevisao(
                ordem.nomeCliente(), ordem.modeloVeiculo(), ordem.placaVeiculo(), ordem.dataProximaRevisao());
    }
}
```

Se já existe lembrete pendente para a OS, atualiza; senão cria um novo com status *pendente*. Helpers de envio e mensagem no fim.

6. Infra — adapters e montagem

Aqui o Spring aparece. Cada classe implementa um port e é um `@Component`. Trocar qualquer uma delas não toca nas camadas internas.

6.1 LogEnviadorDeMensagem

notificacao/infra/LogEnviadorDeMensagem.java

```java
package oficina.mecanica.backendOficina.notificacao.infra;

import oficina.mecanica.backendOficina.notificacao.usecase.port.EnviadorDeMensagem;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Component;

/**
 * Implementação padrão enquanto não há gateway de mensagens contratado:
 * apenas registra a mensagem no log. Para ativar WhatsApp/SMS, basta
 * substituir este bean por outra implementação de {@link EnviadorDeMensagem}.
 */
@Component
public class LogEnviadorDeMensagem implements EnviadorDeMensagem {

    private static final Logger log = LoggerFactory.getLogger(LogEnviadorDeMensagem.class);

    @Override
    public boolean enviar(String telefone, String mensagem, String contexto) {
        log.info("Notificação de {} registrada (envio externo desativado). Para: {} | Texto:
{}",
                contexto, telefone, mensagem);
        return true;
    }
}
```

A implementação atual do envio. Para ativar WhatsApp/SMS, cria-se outra classe que implementa o mesmo port.

6.2 TaskSchedulerAgendadorDeLembrete

notificacao/infra/TaskSchedulerAgendadorDeLembrete.java linhas 1–48

```java
package oficina.mecanica.backendOficina.notificacao.infra;

import oficina.mecanica.backendOficina.notificacao.usecase.port.AgendadorDeLembrete;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.scheduling.TaskScheduler;
import org.springframework.stereotype.Component;

import java.time.LocalDateTime;
import java.time.ZoneId;
import java.util.Map;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ScheduledFuture;

/**
```

```java
 * Adapter do {@link TaskScheduler} do Spring. Mantém em memória quais OS já
 * têm lembrete agendado ou processado nesta execução da aplicação.
 */
@Component
public class TaskSchedulerAgendadorDeLembrete implements AgendadorDeLembrete {

    private static final Logger log =
LoggerFactory.getLogger(TaskSchedulerAgendadorDeLembrete.class);

    private final TaskScheduler taskScheduler;
    private final Map<Long, ScheduledFuture<?>> agendados = new ConcurrentHashMap<>();
    private final Set<Long> processados = ConcurrentHashMap.newKeySet();

    public TaskSchedulerAgendadorDeLembrete(TaskScheduler taskScheduler) {
        this.taskScheduler = taskScheduler;
    }

    @Override
    public boolean estaAgendado(Long ordemServicoId) {
        return ordemServicoId != null && agendados.containsKey(ordemServicoId);
    }

    @Override
    public boolean foiProcessado(Long ordemServicoId) {
        return ordemServicoId != null && processados.contains(ordemServicoId);
    }

    @Override
    public void marcarProcessado(Long ordemServicoId) {
        if (ordemServicoId != null) {
            processados.add(ordemServicoId);
        }
    }
```

Os mapas `agendados` / `processados` que antes viviam no service agora são detalhe deste adapter.

notificacao/infra/TaskSchedulerAgendadorDeLembrete.java — linhas 49–71

```
49      @Override
50      public void agendar(Long ordemServicoId, LocalDateTime quando, Runnable tarefa) {
51          ScheduledFuture<?> future = taskScheduler.schedule(
52                  () -> {
53                      try {
54                          tarefa.run();
55                      } finally {
56                          if (ordemServicoId != null) {
57                              agendados.remove(ordemServicoId);
58                          }
59                      }
60                  },
61                  quando.atZone(ZoneId.systemDefault()).toInstant()
62          );
63
64          if (ordemServicoId != null) {
65              agendados.put(ordemServicoId, future);
66          }
67
68          log.info("Lembrete de revisão preventiva agendado para OS {} em {}", ordemServicoId,
    quando);
69      }
70  }
```

`agendar` envolve o `Runnable` do caso de uso: o `finally` limpa o agendamento após a execução, como antes.

6.3 NotificacaoJpaGateway

Converte `NovaNotificacao` em `NotificacaoModel`. As relações são resolvidas com `getReferenceById` (proxy por id, sem SELECT extra). O `try/catch` que engole falha de persistência existia antes; agora está isolado onde faz sentido — na infra — para não derrubar a finalização da OS.

notificacao/infra/NotificacaoJpaGateway.java linhas 1–44

```java
package oficina.mecanica.backendOficina.notificacao.infra;

import oficina.mecanica.backendOficina.Model.CanalNotificacao;
import oficina.mecanica.backendOficina.Model.NotificacaoModel;
import oficina.mecanica.backendOficina.Model.StatusNotificacao;
import oficina.mecanica.backendOficina.Model.TipoNotificacao;
import oficina.mecanica.backendOficina.Repository.ClienteRepository;
import oficina.mecanica.backendOficina.Repository.NotificacaoRepository;
import oficina.mecanica.backendOficina.Repository.OrdemServicoRepository;
import oficina.mecanica.backendOficina.Repository.VeiculoRepository;
import oficina.mecanica.backendOficina.notificacao.domain.NovaNotificacao;
import oficina.mecanica.backendOficina.notificacao.usecase.port.NotificacaoGateway;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Transactional;

import java.time.LocalDateTime;
import java.util.Optional;

/**
 * Adapter JPA do histórico de notificações. Falhas de persistência são
 * registradas no log e não interrompem o fluxo de negócio que as originou.
 */
@Component
public class NotificacaoJpaGateway implements NotificacaoGateway {

    private static final Logger log = LoggerFactory.getLogger(NotificacaoJpaGateway.class);

    private final NotificacaoRepository notificacaoRepository;
    private final ClienteRepository clienteRepository;
    private final VeiculoRepository veiculoRepository;
    private final OrdemServicoRepository ordemServicoRepository;

    public NotificacaoJpaGateway(NotificacaoRepository notificacaoRepository,
                                 ClienteRepository clienteRepository,
                                 VeiculoRepository veiculoRepository,
                                 OrdemServicoRepository ordemServicoRepository) {
        this.notificacaoRepository = notificacaoRepository;
        this.clienteRepository = clienteRepository;
        this.veiculoRepository = veiculoRepository;
        this.ordemServicoRepository = ordemServicoRepository;
    }
```

Imports de JPA/Spring e os quatro repositórios: este é o único lugar do módulo que conhece `NotificacaoModel`.

notificacao/infra/NotificacaoJpaGateway.java — linhas 45–69

```java
    @Override
    @Transactional
    public Optional<Long> registrar(NovaNotificacao nova) {
        try {
            NotificacaoModel notificacao = new NotificacaoModel();
            notificacao.setCliente(clienteRepository.getReferenceById(nova.clienteId()));
            notificacao.setVeiculo(nova.veiculoId() != null ? veiculoRepository.getReferenceById(nova.veiculoId()) : null);
            notificacao.setOrdemServico(nova.ordemServicoId() != null ? ordemServicoRepository.getReferenceById(nova.ordemServicoId()) : null);
            notificacao.setTipo(nova.tipo());
            notificacao.setAssunto(nova.assunto());
            notificacao.setMensagem(nova.mensagem());
            notificacao.setCanal(CanalNotificacao.whatsapp);
            notificacao.setStatus(nova.status());
            notificacao.setTelefoneDestino(nova.telefoneDestino());
            notificacao.setLida(false);
            notificacao.setDataAgendamento(nova.dataAgendamento());
            notificacao.setDataEnvio(nova.dataEnvio());

            return Optional.ofNullable(notificacaoRepository.save(notificacao).getId());
        } catch (Exception e) {
            log.error("Erro ao registrar notificação no histórico: {}", e.getMessage());
            return Optional.empty();
        }
    }
```

`registrar` : monta a entidade e devolve o id gerado (ou vazio se falhou).

notificacao/infra/NotificacaoJpaGateway.java — linhas 70–100

```java
    @Override
    @Transactional
    public Optional<Long> atualizarLembreteRevisaoPendente(Long ordemServicoId,
                                                           String mensagem,
                                                           LocalDateTime dataAgendamento) {
        return notificacaoRepository
                .findFirstByOrdemServicoIdAndTipoAndStatus(
                        ordemServicoId, TipoNotificacao.revisao_preventiva,
StatusNotificacao.pendente)
                .map(pendente -> {
                    pendente.setMensagem(mensagem);
                    pendente.setDataAgendamento(dataAgendamento);
                    return notificacaoRepository.save(pendente).getId();
                });
    }

    @Override
    @Transactional
    public void registrarResultadoEnvio(Long notificacaoId, boolean enviada, LocalDateTime
dataEnvio) {
        try {
            notificacaoRepository.findById(notificacaoId).ifPresent(notificacao -> {
                notificacao.setStatus(enviada ? StatusNotificacao.enviada :
StatusNotificacao.erro);
                if (enviada) {
                    notificacao.setDataEnvio(dataEnvio);
                }
                notificacaoRepository.save(notificacao);
            });
        } catch (Exception e) {
            log.error("Erro ao atualizar notificação agendada {}: {}", notificacaoId,
e.getMessage());
        }
    }
}
```

`atualizarLembreteRevisaoPendente` e `registrarResultadoEnvio`, ambos `@Transactional`.

6.4 OrdemParaNotificarMapper

notificacao/infra/OrdemParaNotificarMapper.java

```java
package oficina.mecanica.backendOficina.notificacao.infra;

import oficina.mecanica.backendOficina.Model.ClienteModel;
import oficina.mecanica.backendOficina.Model.OrdemServicoModel;
import oficina.mecanica.backendOficina.Model.VeiculoModel;
import oficina.mecanica.backendOficina.notificacao.usecase.OrdemParaNotificar;

/**
 * Converte a entidade JPA no dado de entrada dos casos de uso.
 */
public final class OrdemParaNotificarMapper {

    private OrdemParaNotificarMapper() {
    }

    public static OrdemParaNotificar de(OrdemServicoModel ordemServico) {
        ClienteModel cliente = ordemServico.getCliente();
        VeiculoModel veiculo = ordemServico.getVeiculo();

        return new OrdemParaNotificar(
                ordemServico.getId(),
                cliente != null ? cliente.getId() : null,
                cliente != null ? cliente.getNome() : null,
                cliente != null ? cliente.getTelefone() : null,
                veiculo != null ? veiculo.getId() : null,
                veiculo != null ? veiculo.getModelo() : null,
                veiculo != null ? veiculo.getPlaca() : null,
                ordemServico.getDataProximaRevisao()
        );
    }
}
```

Ponte entre a entidade JPA (mundo externo) e a entrada do caso de uso. Fica na infra porque conhece `OrdemServicoModel`.

6.5 NotificacaoUseCaseConfig — onde tudo se encaixa

Equivale ao `BeanConfiguration` do artigo do Baeldung. É o único lugar que conhece ao mesmo tempo os casos de uso e os adapters. O Spring injeta as implementações dos ports por tipo.

notificacao/infra/NotificacaoUseCaseConfig.java

```java
package oficina.mecanica.backendOficina.notificacao.infra;

import oficina.mecanica.backendOficina.notificacao.usecase.AgendarLembreteRevisaoUseCase;
import oficina.mecanica.backendOficina.notificacao.usecase.NotificarFinalizacaoOrdemUseCase;
import oficina.mecanica.backendOficina.notificacao.usecase.port.AgendadorDeLembrete;
import oficina.mecanica.backendOficina.notificacao.usecase.port.EnviadorDeMensagem;
import oficina.mecanica.backendOficina.notificacao.usecase.port.NotificacaoGateway;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import java.time.Clock;

/**
 * Monta os casos de uso a partir dos adapters. Os casos de uso não conhecem
 * o Spring: toda a ligação com o framework acontece aqui.
 */
@Configuration
public class NotificacaoUseCaseConfig {

    @Bean
    public Clock clock() {
        return Clock.systemDefaultZone();
    }

    @Bean
    public NotificarFinalizacaoOrdemUseCase
notificarFinalizacaoOrdemUseCase(EnviadorDeMensagem enviador,

NotificacaoGateway gateway,
                                                                    Clock clock) {
        return new NotificarFinalizacaoOrdemUseCase(enviador, gateway, clock);
    }

    @Bean
    public AgendarLembreteRevisaoUseCase agendarLembreteRevisaoUseCase(EnviadorDeMensagem
enviador,
                                                                       NotificacaoGateway
gateway,
                                                                       AgendadorDeLembrete
agendador,
                                                                       Clock clock) {
        return new AgendarLembreteRevisaoUseCase(enviador, gateway, agendador, clock);
    }
}
```

Os casos de uso viram beans sem precisar de anotação própria. O `Clock` também é um bean, substituível em teste.

7. Quem chama os casos de uso

Duas portas de entrada diferentes (HTTP via service e cron via scheduler) disparam o mesmo caso de uso. Elas só precisam mapear a entidade para `OrdemParaNotificar` e chamar `executar`.

7.1 OrdemServicoService (finalização da OS)

Service/OrdemServicoService.java — linhas 29–41

```
29      private final OrdemServicoRepository ordemServicoRepository;
30      private final ClienteRepository clienteRepository;
31      private final VeiculoRepository veiculoRepository;
32      private final NotificarFinalizacaoOrdemUseCase notificarFinalizacao;
33      private final AgendarLembreteRevisaoUseCase agendarLembreteRevisao;
34
35      public OrdemServicoService(OrdemServicoRepository ordemServicoRepository,
36                                 ClienteRepository clienteRepository,
37                                 VeiculoRepository veiculoRepository,
38                                 NotificarFinalizacaoOrdemUseCase notificarFinalizacao,
39                                 AgendarLembreteRevisaoUseCase agendarLembreteRevisao) {
40          this.ordemServicoRepository = ordemServicoRepository;
41          this.clienteRepository = clienteRepository;
```

O service passa a depender dos dois casos de uso em vez de `NotificacaoService`.

Service/OrdemServicoService.java — linhas 157–174

```
157     private void adicionarResultadoNotificacoesFinalizacao(OrdemServicoModel ordemServico,
158                                                            OrdemServicoDTOResponse response) {
159         OrdemParaNotificar ordem = OrdemParaNotificarMapper.de(ordemServico);
160
161         ResultadoNotificacao finalizacao = notificarFinalizacao.executar(ordem);
162         response.setMensagemFinalizacaoEnviada(finalizacao.enviada());
163         response.adicionarAviso(finalizacao.aviso());
164
165         if (ordemServico.getDataProximaRevisao() == null) {
166             return;
167         }
168
169         ResultadoNotificacao revisao = agendarLembreteRevisao.executar(ordem);
170         response.setLembreteRevisaoAgendado(revisao.agendada());
171         response.setLembreteRevisaoEnviadoImediatamente(revisao.enviadaImediatamente());
172         response.setDataAgendamentoRevisao(revisao.dataAgendamento());
173         response.adicionarAviso(revisao.aviso());
174     }
```

Uma conversão com o mapper, duas chamadas de `executar`, e o resultado preenche a resposta HTTP.

7.2 RevisaoScheduler (cron diário)

Scheduler/RevisaoScheduler.java — linhas 30–53

```java
    @Scheduled(cron = "${notificacoes.revisao-scheduler-cron:0 0 8 * * *}")
    public void verificarRevisoesProximas() {
        log.info("Scheduler iniciado: verificando revisões preventivas para agendamento.");

        LocalDateTime agora = LocalDateTime.now();
        LocalDateTime janelaInicio = agora.plusDays(7);
        LocalDateTime janelaFim = agora.plusDays(8);

        List<OrdemServicoModel> ordens =
                ordemServicoRepository.findByStatusAndDataProximaRevisaoBetween(
                        StatusOrdemServico.finalizada,
                        janelaInicio,
                        janelaFim
                );

        log.info("Revisões encontradas para lembrete nas próximas 24h: {}", ordens.size());

        for (OrdemServicoModel os : ordens) {
            agendarLembreteRevisao.executar(OrdemParaNotificarMapper.de(os));
        }

        log.info("Scheduler finalizado.");
    }
}
```

O scheduler nem sabe que existe `NotificacaoService`; conhece só o caso de uso.

7.3 O que sobrou em NotificacaoService

O service ficou com o CRUD da tela (listar, marcar lida, contar). O único ponto que ainda envia é o `reenviar` manual, e ele já usa o port em vez de um método privado:

Service/NotificacaoService.java — linhas 24–36

```java
    public NotificacaoService(NotificacaoRepository notificacaoRepository, EnviadorDeMensagem
enviador) {
        this.notificacaoRepository = notificacaoRepository;
        this.enviador = enviador;
    }

    public List<NotificacaoDTOResponse> listar() {
        return notificacaoRepository.listarMaisRecentesPrimeiro()
                .stream()
                .map(this::converterParaResponse)
                .toList();
    }

    public long contarNaoLidas() {
```

De 436 para 137 linhas. Depende do port `EnviadorDeMensagem`, não de uma implementação.

Service/NotificacaoService.java linhas 61–80

```java
    @Transactional
    public NotificacaoDTOResponse reenviar(Long id) {
        NotificacaoModel notificacao = buscarNotificacao(id);
        ClienteModel cliente = notificacao.getCliente();
        String telefone = cliente != null ? cliente.getTelefone() :
notificacao.getTelefoneDestino();

        String destinatario = Telefone.normalizar(telefone);
        boolean enviada = destinatario != null
                && enviador.enviar(destinatario, notificacao.getMensagem(), "reenvio da
notificação " + id);

        notificacao.setStatus(enviada ? StatusNotificacao.enviada : StatusNotificacao.erro);
        notificacao.setTelefoneDestino(telefone);
        notificacao.setLida(false);

        if (enviada) {
            notificacao.setDataEnvio(LocalDateTime.now());
        }

        return converterParaResponse(notificacaoRepository.save(notificacao));
    }
```

`Telefone.normalizar` + `enviador.enviar`: as mesmas peças do domínio e do port.

8. O ganho na prática: teste sem Spring

Antes, testar a regra dos 7 dias exigia contexto Spring, banco, `TaskScheduler` e relógio real. Agora o caso de uso é instanciado à mão com *fakes* dos ports e um `Clock.fixed`. Os 6 testes rodam em 22 ms.

test/notificacao/usecase/AgendarLembreteRevisaoUseCaseTest.java — linhas 26–74

```java
class AgendarLembreteRevisaoUseCaseTest {

    private static final LocalDateTime AGORA = LocalDateTime.of(2026, 9, 15, 10, 0);

    private EnviadorFake enviador;
    private GatewayFake gateway;
    private AgendadorFake agendador;
    private AgendarLembreteRevisaoUseCase useCase;

    @BeforeEach
    void setUp() {
        enviador = new EnviadorFake();
        gateway = new GatewayFake();
        agendador = new AgendadorFake();
        Clock clock = Clock.fixed(AGORA.toInstant(ZoneOffset.UTC), ZoneId.of("UTC"));
        useCase = new AgendarLembreteRevisaoUseCase(enviador, gateway, agendador, clock);
    }

    @Test
    void revisaoDistanteAgendaLembreteSeteDiasAntes() {
        OrdemParaNotificar ordem = ordem(AGORA.plusDays(30));

        ResultadoNotificacao resultado = useCase.executar(ordem);

        assertTrue(resultado.agendada());
        assertFalse(resultado.enviada());
        assertEquals(AGORA.plusDays(23), resultado.dataAgendamento());
        assertEquals(AGORA.plusDays(23), agendador.quando);
        assertEquals(1, gateway.registradas.size());
        assertEquals(StatusNotificacao.pendente, gateway.registradas.get(0).status());
        assertTrue(enviador.enviadas.isEmpty());
    }

    @Test
    void revisaoDentroDoPrazoEnviaImediatamente() {
        OrdemParaNotificar ordem = ordem(AGORA.plusDays(3));

        ResultadoNotificacao resultado = useCase.executar(ordem);

        assertTrue(resultado.enviadaImediatamente());
        assertFalse(resultado.agendada());
        assertEquals(1, enviador.enviadas.size());
        assertEquals("11999998888", enviador.enviadas.get(0));
        assertEquals(StatusNotificacao.enviada, gateway.registradas.get(0).status());
        assertEquals(AGORA, gateway.registradas.get(0).dataEnvio());
        assertTrue(agendador.processados.contains(1L));
    }
```

`setUp` monta o caso de uso sem framework. Cada teste verifica um caminho da regra e o que chegou em cada port.

test/notificacao/usecase/AgendarLembreteRevisaoUseCaseTest.java — linhas 99–122

```
99      @Test
100     void tarefaAgendadaEnviaEAtualizaNotificacaoPendente() {
101         useCase.executar(ordem(AGORA.plusDays(30)));
102
103         agendador.tarefa.run();
104
105         assertEquals(1, enviador.enviadas.size());
106         assertEquals(Long.valueOf(100L), gateway.resultadoId);
107         assertTrue(gateway.resultadoEnviada);
108         assertTrue(agendador.processados.contains(1L));
109     }
110
111     @Test
112     void naoReagendaQuandoJaExisteAgendamento() {
113         agendador.agendados.add(1L);
114
115         ResultadoNotificacao resultado = useCase.executar(ordem(AGORA.plusDays(30)));
116
117         assertTrue(resultado.agendada());
118         assertEquals("Lembrete de revisão já estava agendado.", resultado.mensagem());
119         assertTrue(gateway.registradas.isEmpty());
120         assertNull(agendador.tarefa);
121     }
```

O teste executa o `Runnable` capturado pelo agendador fake — simulando o dia do lembrete sem esperar.

test/notificacao/usecase/AgendarLembreteRevisaoUseCaseTest.java linhas 128–161

```java
    private static class EnviadorFake implements EnviadorDeMensagem {
        final List<String> enviadas = new ArrayList<>();

        @Override
        public boolean enviar(String telefone, String mensagem, String contexto) {
            enviadas.add(telefone);
            return true;
        }
    }

    private static class GatewayFake implements NotificacaoGateway {
        final List<NovaNotificacao> registradas = new ArrayList<>();
        Long resultadoId;
        boolean resultadoEnviada;

        @Override
        public Optional<Long> registrar(NovaNotificacao notificacao) {
            registradas.add(notificacao);
            return Optional.of(100L);
        }

        @Override
        public Optional<Long> atualizarLembreteRevisaoPendente(Long ordemServicoId, String mensagem,
                                                               LocalDateTime dataAgendamento) {
            return Optional.empty();
        }

        @Override
        public void registrarResultadoEnvio(Long notificacaoId, boolean enviada, LocalDateTime dataEnvio) {
            resultadoId = notificacaoId;
            resultadoEnviada = enviada;
        }
    }
```

Fakes: implementações mínimas dos ports que só guardam o que receberam. Sem Mockito, sem banco.

9. Resumo do fluxo e próximos passos

Passo	Camada	Código
1. OS é finalizada (PUT) ou cron roda	entrada	`OrdemServicoService` / `RevisaoScheduler`
2. Entidade vira input	infra	`OrdemParaNotificarMapper.de(os)`
3. Regra decide o caminho	usecase	`AgendarLembreteRevisaoUseCase.executar`
4. Texto e telefone	domain	`MensagemNotificacao`, `Telefone`
5. Envia / agenda / persiste	port → infra	`EnviadorDeMensagem`, `AgendadorDeLembrete`, `NotificacaoGateway`
6. Resultado volta à entrada	usecase	`ResultadoNotificacao`

Para ligar o Twilio

Criar `TwilioEnviadorDeMensagem implements EnviadorDeMensagem` em `infra`, anotar com `@Component` e `@Primary` (ou condicionar por property). Nenhuma linha de `domain` ou `usecase` muda — e o teste unitário continua passando igual.

> **Decisões conscientes:** (1) os enums de `Model` foram reutilizados no domínio por serem Java puro; (2) o comportamento foi preservado, inclusive o `try/catch` de persistência, agora isolado no gateway; (3) aplicou-se Clean Architecture só neste bloco — o restante do backend continua em camadas técnicas, por escolha de escopo.